

E-COMMERCE DATABASE

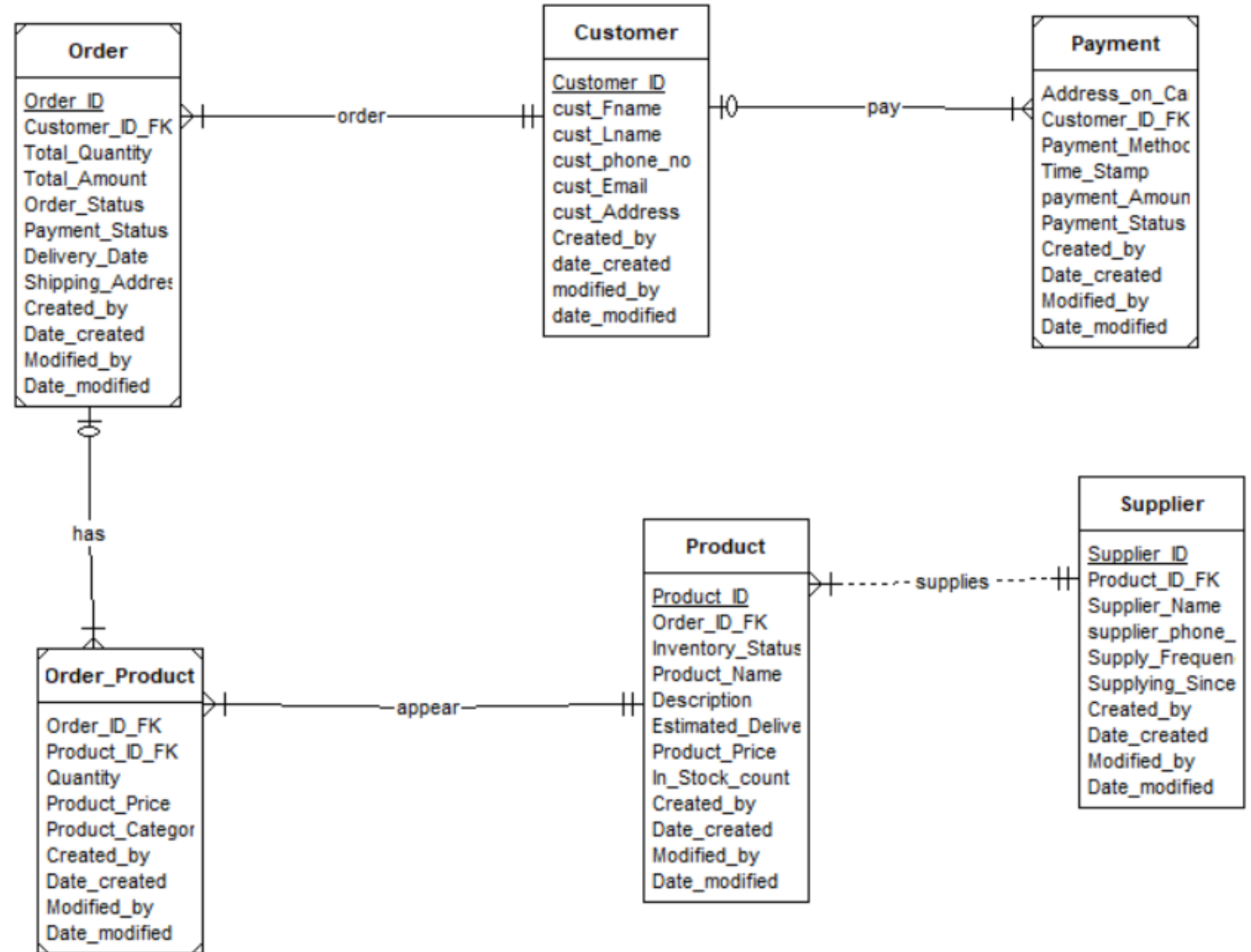


INTRODUCTION

E-commerce databases provide a gateway to unparalleled insights, transforming raw information into strategic advantages.

This aim to support business processes, to maintain data integrity between object: Customer, Order, Product, Payment, Supplier

ERD DIAGRAM



SEQUENCES AND TRIGGERS

SEQUENCES: to generate values starting from 1000 and increasing by 1 each time for Customer_ID field. Sequences created for each table start with 1000, 2000, 3000, 4000, 5000

```
CREATE SEQUENCE Customer_id_seq
start with 1000
increment by 1
NO MAXVALUE
MINVALUE 1000;
```

TRIGGERS: to establish an automatic incrementing mechanism for the Customer_ID field. When a new row is inserted into the Customer table, the trigger calls the function, which, in turn, assigns the next sequence value to the Customer_ID field of the newly inserted row.

```
CREATE OR REPLACE FUNCTION Customer_id_auto_increment()
RETURNS TRIGGER AS $$
BEGIN
    NEW.Customer_ID = NEXTVAL('Customer_id_seq');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Customer_id_auto_increment_trigger
BEFORE INSERT ON Customer
FOR EACH ROW
EXECUTE FUNCTION Customer_id_auto_increment();

-- Creating Trigger and Trigger Function for Order Table
CREATE OR REPLACE FUNCTION Order_id_auto_increment()
RETURNS TRIGGER AS $$
BEGIN
    NEW.Order_ID = NEXTVAL('Order_id_seq');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

ADVANCES QUERIES and OUTPUTS

pgAdmin 4

File Object Tools Help

Object Explorer

- > Casts
- > Catalogs
- > Event Triggers
- > Extensions
- > Foreign Data Wrappers
- > Languages
- > Publications
- > Schemas (1)
 - public
 - > Aggregates
 - > Collations
 - > Domains
 - > FTS Configurations
 - > FTS Dictionaries
 - > FTS Parsers
 - > FTS Templates
 - > Foreign Tables
 - > Functions
 - > Materialized Views
 - > Operators
 - > Procedures
 - > 1.3 Sequences (5)
 - 1.3 customer_id_seq
 - 1.3 order_id_seq
 - 1.3 payment_id_seq
 - 1.3 product_id_seq
 - 1.3 supplier_id_seq
 - > Tables (6)
 - > customer
 - > orders

Dashboard Properties SQL Statistics Dependencies Dependents Processes project/postgres@PostgreSQL 16*

project/postgres@PostgreSQL 16

No limit

Query Query History

```
45 --Advanced Query 1: Supplier Reliability Report
46 ---This query generates a report on supplier reliability by comparing the estimated and actual de
47 SELECT
48     s.Supplier_Name,
49     p.Product_Name,
50     p.Estimated_Delivery_Date,
51     MIN(o.Delivery_Date) AS Actual_Delivery_Date,
52     CASE
53         WHEN MIN(o.Delivery_Date) <= p.Estimated_Delivery_Date THEN 'On Time'
54         ELSE 'Delayed'
55     END AS Delivery_Status
56 FROM Supplier s
```

Data Output Messages Notifications

	supplier_name character varying (50)	product_name character varying (50)	estimated_delivery_date date	actual_delivery_date date	delivery_status text
1	Acme Corp	Widget A	2023-12-25	2023-12-15	On Time
2	Apple Inc.	Apple iPhone 14 Pro	2023-11-04	2023-10-31	On Time
3	Initech LLC	Appliance D	2023-12-20	2023-12-15	On Time
4	Intel LLC	Drone	2023-12-20	2023-10-31	On Time
5	Shipping and Co	Kids book	2023-12-25	2023-10-31	On Time

Total rows: 5 of 5 Query complete 00:00:00.068

60°F Cloudy

Search

Taskbar icons: Microsoft Store, Search, Edge, File Explorer, Task View, Teams, OneDrive, Word, Outlook, etc.

ADVANCES QUERIES and OUTPUTS

pgAdmin 4

File Object Tools Help

Object Explorer

- > Casts
- > Catalogs
- > Event Triggers
- > Extensions
- > Foreign Data Wrappers
- > Languages
- > Publications
- > Schemas (1)
 - public
 - > Aggregates
 - > Collations
 - > Domains
 - > FTS Configurations
 - > FTS Dictionaries
 - > FTS Parsers
 - > FTS Templates
 - > Foreign Tables
 - > Functions
 - > Materialized Views
 - > Operators
 - > Procedures
 - > 1.3 Sequences (5)
 - 1.3 customer_id_seq
 - 1.3 order_id_seq
 - 1.3 payment_id_seq
 - 1.3 product_id_seq
 - 1.3 supplier_id_seq
 - > Tables (6)
 - > customer
 - > orders

Dashboard Properties SQL Statistics Dependencies Dependents Processes project/postgres@PostgreSQL 16

Query Query History

```
--Advanced Query 2: Customer Spending Analysis by Payment Method
SELECT
  c.cust_Fname || ' ' || c.cust_LName AS Customer_Name,
  p.Payment_Method,
  COUNT(*) AS Number_of_Orders,
  SUM(p.Payment_Amount) AS Total_Spent,
  CASE
    WHEN SUM(p.Payment_Amount) > 500 THEN 'High Spender'
    WHEN SUM(p.Payment_Amount) BETWEEN 200 AND 500 THEN 'Medium Spender'
    ELSE 'Low Spender'
  END AS Spending_Category
```

Data Output Messages Notifications

	customer_name text	payment_method character varying (50)	number_of_orders bigint	total_spent bigint	spending_category text
1	Bob Brown	PayPal	1	500	Medium Spender
2	Cobber Bron	PayPal	1	500	Medium Spender
3	Charlie Davis	Visa	1	275	Medium Spender
4	Jane Smith	Visa	1	200	Medium Spender
5	John Doe	Credit Card	1	100	Low Spender
6	Alex Bay	Credit Card	1	100	Low Spender
7	Cobb Brown	Visa	1	75	Low Spender
8	Alice Johnson	MasterCard	1	50	Low Spender
9	Sophie Hightower	MasterCard	1	50	Low Spender
10	Slesha Higher	Visa	1	0	Low Spender

Total rows: 10 of 10 Query complete 00:00:00.090

60°F Cloudy

Search

CHALLENGES

- Challenge about running Trigger to pull data from multi-table
- Challenges in meeting group and working together due to difference schedule
- Ensuring that we were able to complete the deliverables in a timely manner, in accordance to everyone's schedule
- Figuring out how to create database in PostgreSQL and ensure all our queries ran successfully.

THANK YOU !

